MODULE *compact_relay*

Compact block relay reconstruction specification.

Models the draft's "Compact Block Relay" machinery between a sender S (who has the new block) and a requester R (who holds some of its transactions): compact block announcements carrying a per-announcement nonce, short-ID matching against R's *mempool*, *get − tx SHORTID requests for the missing*
transactions, *and the reconstruction outcome.*

The stateful subtlety is the nonce : short transaction IDs are relative to
one compact block s nonce, and the responding node interprets *SHORTID* references "using the nonce of the compact block it most recently sent to the requesting peer for the identified block" ("Requesting Missing Transactions"). A sender may legitimately announce the same block more than once with a fresh nonce — for example when re-announcing its tip on a replacement announcement stream (see Finding 2 in .. /documents/v2-*modeling.md*). *SHORTID* requests computed from an earlier announcement then arrive under a newer nonce: each reference matches no transaction (not-found) or, by 48-bit collision, a wrong one. The model resolves that choice adversarially.

Announcements are best-effort: under backpressure the sender may drop one rather than queue it ("Announcement Streams"), and a reset replacement stream loses records in flight. The model lets a re-announcement be lost while the requester is mid-attempt — after which every reference the requester can ever send is stale, and the compact path alone cannot converge.

The draft's escape hatch is reconstruction failure handling: the node "SHOULD fall back to requesting the full block via get-blocks, and MUST NOT assign a misbehavior penalty solely because reconstruction failed". Two switches probe it:

Fallback	= FALSE →	reconstruction failure retries the compact path instead of falling back. Liveness (*EventuallyHasBlock*) is violated: announce budgets end and the requester waits forever. The SHOULD is load-bearing — no other rule guarantees the block arrives.
PenalizeReconFail	= TRUE →	reconstruction failure scores 20 points, against the MUST NOT. *NoHonestPenalty* is violated: honest nonce churn frames the sender.

With *Fallback* = TRUE and no penalties, every property holds under maximal nonce churn, and *WrongTxNeverAccepted* states the draft's own claim that short-ID matching cannot weaken consensus: a wrongly matched transaction fails the merkle check and never enters an accepted block.

See .. /documents/v2-*modeling.md* for the full write-up.

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT *NTx*	transactions in the block: 1 .. *NTx*
CONSTANT *MaxAnnounce*	compact block announcements available to S
CONSTANT *Fallback*	see module comment
CONSTANT *PenalizeReconFail*	see module comment

$Txs \triangleq 1 \,..\, NTx$

VARIABLES

mempool,	R: subset of *Txs* held before the block arrives
has_block,	R: TRUE once the block is accepted (fully reconstructed or fetched)
score,	R's misbehavior score for S (MUST stay 0)

ann_q,	announcement stream toward R: sequence of nonces (in order)
$last\_sent$,	S: nonce of the compact block most recently sent to R ($0 = none$)
$announced$,	S: announcements used
via,	how R got the block: "none" | "match" | "recon" | "full" (ghost)
req,	$SHORTID$ get-tx toward S: [active, nonce, missing]
$resp$,	get-tx response toward R: [active, ok : txs served correctly, bad : a not-found or wrong tx occurred]
fb	fallback get-blocks state: "none" | "requested" | "served"

$vars \triangleq \langle mempool, has\_block, score, ann\_q, last\_sent, announced, via, req, resp, fb \rangle$

$NoReq \triangleq [active \mapsto \text{FALSE}, nonce \mapsto 0, missing \mapsto \{\}]$
$NoResp \triangleq [active \mapsto \text{FALSE}, ok \mapsto \{\}, bad \mapsto \text{FALSE}]$

$Init \triangleq$
- $\land\ mempool \in \text{SUBSET}\ Txs$
- $\land\ has\_block = \text{FALSE}$
- $\land\ score = 0$
- $\land\ ann\_q = \langle\rangle$
- $\land\ via =$ "none"
- $\land\ last\_sent = 0$
- $\land\ announced = 0$
- $\land\ req = NoReq$
- $\land\ resp = NoResp$
- $\land\ fb =$ "none"

S announces the block as a compact block with a fresh nonce — the first announcement, or a legitimate re-announcement (replacement stream, *BIP* 152 high-bandwidth relay). Nonces are the announcement number.

$Announce \triangleq$
- $\land\ announced < MaxAnnounce$
- $\land\ announced' = announced + 1$
- $\land\ ann\_q' = Append(ann\_q, announced + 1)$
- $\land\ last\_sent' = announced + 1$
- $\land\ \text{UNCHANGED}\ \langle mempool, has\_block, score, via, req, resp, fb \rangle$

R processes the next compact block announcement: transactions it holds match by short ID; the missing ones are requested by $SHORTID$ reference, remembering which nonce they were computed under. If nothing is missing the block is reconstructed immediately. Announcements for a block already held are ignored.

$RecvCompactBlock \triangleq$
- $\land\ ann\_q \neq \langle\rangle$
- $\land\ \neg req.active \land \neg resp.active \land fb =$ "none"
- $\land\ ann\_q' = Tail(ann\_q)$

$\land$ IF has_block
  THEN UNCHANGED $\langle mempool, has\_block, score, last\_sent, announced, via, req, resp, fb\rangle$
  ELSE IF $Txs \subseteq mempool$
  THEN $\land\ has\_block' = \text{TRUE}$
       $\land\ via' = \text{“match”}$
       $\land$ UNCHANGED $\langle mempool, score, last\_sent, announced, req, resp, fb\rangle$
  ELSE $\land\ req' = [active \mapsto \text{TRUE}, nonce \mapsto Head(ann\_q), missing \mapsto Txs \setminus mempool]$
       $\land$ UNCHANGED $\langle mempool, has\_block, score, last\_sent, announced, via, resp, fb\rangle$

A queued re-announcement is lost while the requester is busy with an attempt: announcements are best-effort under backpressure, and a reset replacement stream drops records in flight. Adversarial, never forced.

$DropAnnouncement \triangleq$
  $\land\ ann\_q \neq \langle\rangle$
  $\land\ req.active \lor resp.active$
  $\land\ ann\_q' = Tail(ann\_q)$
  $\land$ UNCHANGED $\langle mempool, has\_block, score, last\_sent, announced, via, req, resp, fb\rangle$

S serves the *SHORTID* references, interpreting them under the nonce of the compact block it most recently sent. Fresh references resolve exactly; stale ones (an announcement has happened since) match no transaction or, by collision, a wrong one — resolved adversarially per reference.

$ServeGetTx \triangleq$
  $\land\ req.active$
  $\land\ \exists\, badset \in \text{SUBSET}\ req.missing :$
       $\land\ (req.nonce = last\_sent) \Rightarrow badset = \{\}$
       $\land\ (req.nonce \neq last\_sent) \Rightarrow badset \neq \{\}$
       $\land\ resp' = [active \mapsto \text{TRUE}, ok \mapsto req.missing \setminus badset, bad \mapsto badset \neq \{\}]$
  $\land\ req' = NoReq$
  $\land$ UNCHANGED $\langle mempool, has\_block, score, ann\_q, last\_sent, announced, via, fb\rangle$

R completes reconstruction from a clean response: the block passes its merkle check and is accepted.

$RecvTxsComplete \triangleq$
  $\land\ resp.active$
  $\land\ \neg resp.bad$
  $\land\ mempool' = mempool \cup resp.ok$
  $\land\ has\_block' = \text{TRUE}$
  $\land\ via' = \text{“recon”}$
  $\land\ resp' = NoResp$
  $\land$ UNCHANGED $\langle score, ann\_q, last\_sent, announced, req, fb\rangle$

A not-found result or a wrong transaction makes reconstruction fail (a wrong transaction fails the merkle check). What happens next is the switch: fall back to get-blocks as the draft says SHOULD, or retry the compact path with the next announcement; and, forbidden by the draft's

MUST NOT, a penalty may be scored.

$RecvTxsFailed \triangleq$

$\land\ resp.active$
$\land\ resp.bad$
$\land\ mempool' = mempool \cup resp.ok$
$\land\ resp' = NoResp$
$\land\ fb' = \text{IF } Fallback \text{ THEN } \text{“requested”} \text{ ELSE } \text{“none”}$
$\land\ score' = \text{IF } PenalizeReconFail \text{ THEN } score + 20 \text{ ELSE } score$
$\land\ \text{UNCHANGED } \langle has\_block,\ ann\_q,\ last\_sent,\ announced,\ via,\ req \rangle$

The fallback get-blocks round trip: S always holds the full block.

$ServeFallback \triangleq$

$\land\ fb = \text{“requested”}$
$\land\ fb' = \text{“served”}$
$\land\ \text{UNCHANGED } \langle mempool,\ has\_block,\ score,\ ann\_q,\ last\_sent,\ announced,\ via,\ req,\ resp \rangle$

$RecvFallback \triangleq$

$\land\ fb = \text{“served”}$
$\land\ has\_block' = \text{TRUE}$
$\land\ via' = \text{“full”}$
$\land\ fb' = \text{“none”}$
$\land\ \text{UNCHANGED } \langle mempool,\ score,\ ann\_q,\ last\_sent,\ announced,\ req,\ resp \rangle$

$Next \triangleq$

$\lor\ Announce$
$\lor\ RecvCompactBlock$
$\lor\ DropAnnouncement$
$\lor\ ServeGetTx$
$\lor\ RecvTxsComplete$
$\lor\ RecvTxsFailed$
$\lor\ ServeFallback$
$\lor\ RecvFallback$

Everything a peer must eventually do is weakly fair; *Announce* is fair too, so the block is announced at all, and its budget bounds the churn. *DropAnnouncement* carries no fairness: it is adversarial, never forced — *WF* on the other actions cannot force it either, since *RecvCompactBlock* is enabled whenever it is.

$Spec \triangleq$

$Init$
$\land\ \Box[Next]_{vars}$
$\land\ \mathrm{WF}_{vars}(Announce)$
$\land\ \mathrm{WF}_{vars}(RecvCompactBlock)$
$\land\ \mathrm{WF}_{vars}(ServeGetTx)$

$\land \mathrm{WF}_{vars}(RecvTxsComplete)$
$\land \mathrm{WF}_{vars}(RecvTxsFailed)$
$\land \mathrm{WF}_{vars}(ServeFallback)$
$\land \mathrm{WF}_{vars}(RecvFallback)$

Liveness: R eventually holds the block.

$EventuallyHasBlock \triangleq \Diamond has\_block$

Safety invariants.

$TypeOK \triangleq$
- $\land \quad mempool \subseteq Txs$
- $\land \quad has\_block \in \text{BOOLEAN}$
- $\land \quad score \in Nat$
- $\land \quad last\_sent \in 0 \,..\, MaxAnnounce$
- $\land \quad announced \in 0 \,..\, MaxAnnounce$
- $\land \quad fb \in \{\text{“none”}, \text{“requested”}, \text{“served”}\}$
- $\land \quad via \in \{\text{“none”}, \text{“match”}, \text{“recon”}, \text{“full”}\}$

The draft's MUST NOT: reconstruction failure carries no penalty.

$NoHonestPenalty \triangleq score = 0$

The draft's consensus claim: short-ID matching cannot weaken consensus. A block accepted through reconstruction was assembled from the full, correct transaction set (bad responses never reach the *mempool* and fail the merkle check instead); anything else arrived as a full block.

$WrongTxNeverAccepted \triangleq$
$has\_block \land via \in \{\text{“match”}, \text{“recon”}\} \Rightarrow Txs \subseteq mempool$